Online Quiz and Exam System

Software Specification (SDD)

Version 1.0 | 29 October 2026 | Draft specification

Tech stack: UI - React JS | API - FastAPI | DB - PostgreSQL

Flow: Spec -> Plan -> Implement -> Test -> Review

0. How to use this document (for the AI agent)

This document is the single source of truth. An AI coding agent shall build the application strictly from it.

- Build in the order given in Section 11 (Tasks 1 to 6). Finish and test one task before starting the next.
- Every requirement has an ID (S, P, FR, UI, API, NF). Reference IDs in code comments and test names.
- If something is not specified, choose the simplest option, do not add features, and list the choice in a file DECISIONS.md.
- A chat prompt never overrides this document. If a change is needed, update this document first, then the code.
- Final output: working application, automated tests, a test result table (Section 10) and a short review note.

1. Scope and baseline assumptions

1.1 Purpose and intended outcome

Build a web application where teachers create multiple-choice quizzes and students attempt them. The system scores each attempt automatically and shows results. Teachers see the results of their own quizzes.

Source requirement: "Teachers create quizzes with questions, students attempt quizzes, the system auto-scores and shows results."

1.2 How to read this specification

Requirements marked **S** come directly from the source requirement and the assignment. Requirements marked **P** are proposed elaborations that make the brief buildable and testable. "Shall" describes required behavior.

ID	Basis	Requirement
S01	S	Teachers shall be able to create quizzes with questions and answer options.
S02	S	Students shall be able to view and attempt published quizzes.
S03	S	The system shall score attempts automatically.
S04	S	Students shall be able to view their results; teachers shall be able to view results of their quizzes.
S05	S	Required fields shall be validated and permissions enforced.
S06	S	The system shall provide a web UI (React JS) and a REST API (FastAPI) backed by PostgreSQL.
S07	S	Every new quiz shall start with status Draft.
S08	S	Verification shall include valid, invalid and unauthorized test cases.

1.3 Proposed assumptions

- **A01** One organization. Each account has exactly one role: Teacher or Student. Accounts are created by a seed script; there is no user-management screen. Login uses email and password with a JWT token.
- **A02** Quiz statuses: Draft, Published, Closed. Attempt statuses: In Progress, Submitted.
- **A03** Questions are multiple choice with exactly one correct option. Every question is worth 1 mark. Score = number of correct answers.
- **A04** Each student may attempt each quiz once.
- **A05** There is no time limit per quiz in this release.

1.4 Included in the initial release

Login, quiz and question management, publishing and closing, student attempts, auto-scoring, student results, teacher results per quiz, scoped lists, server-side permissions.

1.5 Excluded from this release

Timers and auto-submit, question types other than multiple choice with one answer, negative marking, question banks, random question order, retakes, email or push notifications, file or image uploads, user registration screens, password reset, certificates, proctoring, reports and charts, multi-organization tenancy, AI features.

2. Actors and access control

P01 Every API operation except login and health shall require an authenticated, active account. The server derives user ID and role from the validated JWT. Client-supplied identity or role values shall never decide access.

Capability	Teacher	Student
Create quiz and questions	Yes	No
Edit own quiz while Draft	Yes	No
Publish or close own quiz	Yes	No
List quizzes	Own quizzes (all statuses)	Published quizzes only
View quiz questions with correct answers	Own quizzes	Never (before or during attempt)
Start and submit an attempt	No	Yes, once per quiz
View own attempt result	N/A	Yes, own only
View all results of a quiz	Own quizzes only	No
Delete a quiz	Own quiz while Draft	No

Authorization rules

- **P02** Quiz ownership is the immutable teacher_id set at creation. A teacher sees and changes only their own quizzes. A student sees only Published quizzes, and only their own attempts.
- **P03** Missing or expired token returns 401. A valid user using a role-restricted endpoint returns 403. A nonexistent resource and a resource the user may not read return the same 404.
- **P04** Correct answers shall never be sent to a student before that student has submitted the attempt.
- **P05** Inactive accounts cannot log in.

Primary user journeys

Teacher: sign in -> New Quiz -> enter title and description -> add questions with options and mark the correct one -> Publish -> later view Results -> Close the quiz.

Student: sign in -> Available Quizzes -> open a quiz -> Start Attempt -> answer questions -> Submit -> see score and per-question review -> find it again in My Results.

3. Functional requirements

Quizzes and questions

- **FR01** [S01, S07] On valid submission the server shall create a quiz with a UUID, the authenticated teacher as owner, status Draft, and server timestamps (UTC).
- **FR02** [S05] Title and description shall pass the rules in Section 6. Invalid or unknown fields shall fail with 422 and create nothing.
- **FR03** [S01] A teacher shall add, edit and remove questions on a Draft quiz. Each question has text and 2 to 6 options, exactly one marked correct.
- **FR04** [P] A teacher shall edit the title and description of a Draft quiz. Published and Closed quizzes are read-only.

- **FR05** [P] A teacher shall delete a Draft quiz. Published and Closed quizzes cannot be deleted.
- **FR06** [P] Publishing requires at least 1 question. Every question must satisfy the option rules at publish time.

Attempts and scoring

- **FR07** [S02] A student shall see the list of Published quizzes with title, description and question count, and whether they have already attempted it.
- **FR08** [S02] A student shall start an attempt on a Published quiz. The response contains the questions and options without the correct flag. A second attempt on the same quiz shall be rejected with 409.
- **FR09** [S02, S03] A student shall submit answers as a list of question and option pairs. Unanswered questions are allowed and score 0. The server shall validate that each option belongs to its question.
- **FR10** [S03] On submit the server shall compute score (correct answers), total (number of questions) and percentage (rounded to 1 decimal), store them, and mark the attempt Submitted, all in one transaction.
- **FR11** [P] A submitted attempt cannot be changed or submitted again.

Results

- **FR12** [S04] A student shall view score, total, percentage, submit time and a per-question review (selected option, correct option, correct or incorrect) for their own submitted attempts. My Results lists all of them, newest first.
- **FR13** [S04] A teacher shall view, for each own quiz, the list of attempts (student name, score, total, percentage, submitted time) and summary figures: attempt count, average percentage, highest and lowest score.
- **FR14** [P] Closing a quiz stops new attempts. Students who already submitted keep their results. An In Progress attempt on a quiz that gets closed may still be submitted once.

4. Lifecycle and business rules

Quiz lifecycle

From	To	Conditions
New	Draft	Server-controlled; never supplied by the client.
Draft	Published	Owner teacher; at least 1 valid question.
Published	Closed	Owner teacher.
Draft	Deleted	Owner teacher; removes its questions and options.

- **P06** All other quiz transitions are rejected with 409 INVALID_TRANSITION. Closed cannot be reopened. Published cannot return to Draft.
- **P07** Question and option changes are accepted only while the quiz is Draft, otherwise 409 QUIZ_LOCKED.

Attempt lifecycle

From	To	Conditions
New	In Progress	Student; quiz Published; no earlier attempt by this student on this quiz.
In Progress	Submitted	Owner student; valid answers; score computed.

- **P08** Only one attempt per student per quiz, enforced by a database unique constraint on (quiz_id, student_id).
- **P09** Score is computed only on the server from stored correct flags. Client-provided scores are rejected as unknown fields.
- **P10** Attempt creation, answer storage and score computation commit atomically. A failure leaves no partial result.

Example scenario

A teacher creates "Python Basics" with 5 questions and publishes it. Student A starts an attempt, answers 4 of 5 questions, 3 of them correctly, and submits. The system shows score 3 of 5 (60.0%). Student A cannot attempt again. The teacher sees one attempt with an average of 60.0%.

5. Web interface specification

Screen	Role	Required content and behavior
Sign in	All	Email and password. Redirect to the role's home page after login.
My Quizzes	Teacher	Own quizzes with title, status, question count, created date. New Quiz action.
Quiz Editor	Teacher	Title, description, list of questions with options, mark-correct control, add and remove question, Publish and Close and Delete actions according to status.
Quiz Results	Teacher	Summary figures and attempts table.
Available Quizzes	Student	Published quizzes with Start or Attempted label.
Take Quiz	Student	One question list with radio options and a Submit button. Unanswered questions are marked before submit with a confirmation.
Result Detail	Student	Score, percentage and per-question review.
My Results	Student	All own submitted attempts.

Form behavior

- **UI01** Required fields are visibly labelled. Whitespace-only input fails validation. Client validation mirrors server rules; the server stays authoritative. Entered text is retained on errors.
- **UI02** The UI shall disable Submit while a request is pending and never show success before the server confirms.
- **UI03** Empty lists show "No quizzes found" (or "No results yet"). Loading indicators appear while loading. Errors offer Retry and never show stack traces.
- **UI04** 401 returns to sign in. 403 shows "This action is not available." 404 shows "Not found or unavailable." 409 shows the server message.
- **UI05** Navigation: Teacher sees My Quizzes and New Quiz. Student sees Available Quizzes and My Results. A menu item for an excluded feature (such as Reports) shall not appear.
- **UI06** Controls are usable by keyboard, with visible focus and labels. Status is shown as text, not only color. Layout works at 375 px and 1440 px widths without page-level horizontal scrolling.
- **UI07** Dates are stored in UTC and shown in the browser timezone.

6. Data model and validation

All IDs are UUIDs. Text length counts characters after trimming. Timestamps are UTC.

Field rules

Entity.field	Rule
quiz.title	Required, 5 to 150 characters.
quiz.description	Required, 10 to 1,000 characters.
quiz.status	Draft, Published, Closed. Default Draft.
question.text	Required, 5 to 500 characters.
option.text	Required, 1 to 200 characters.
options per question	2 to 6, exactly one with is_correct = true.
questions per quiz	1 to 50 (1 needed to publish).
user.email	Valid email, unique.
user.role	Teacher or Student.

Tables

- **users**: id, name, email (unique), password_hash, role, is_active, created_at
- **quizzes**: id, teacher_id (FK users), title, description, status, created_at, updated_at, published_at, closed_at

- **questions**: id, quiz_id (FK, cascade delete), text, position, created_at
- **options**: id, question_id (FK, cascade delete), text, is_correct, position
- **attempts**: id, quiz_id (FK), student_id (FK), status, score, total, percentage, started_at, submitted_at. Unique (quiz_id, student_id).
- **attempt_answers**: id, attempt_id (FK), question_id (FK), option_id (FK, nullable). Unique (attempt_id, question_id).

Persistence constraints

Enforce enums, required fields and foreign keys in the database. Add indexes on quizzes(teacher_id, status), attempts(student_id), attempts(quiz_id). Partial unique index so each question has at most one correct option. Use Alembic migrations. There is no deletion of attempts or users.

7. REST API contract

Base path /api/v1. JSON requests and responses. Lists return {items, page, page_size, total} with defaults page=1, page_size=20, maximum 100. Invalid parameters return 422.

Method and path	Role	Contract
POST /auth/login	Public	Body: email, password. Returns 200 with access_token and user (id, name, role). 401 on wrong credentials.
GET /auth/me	Any	Returns current user.
POST /quizzes	Teacher	Body: title, description. 201, status Draft.
GET /quizzes	Any	Teacher: own quizzes, optional status filter and q (title search). Student: Published only.
GET /quizzes/{id}	Any	Teacher owner: full quiz with correct flags. Student: Published quiz summary only, no questions.
PATCH /quizzes/{id}	Teacher	Body: title and/or description. Draft only.
DELETE /quizzes/{id}	Teacher	Draft only. 204.
POST /quizzes/{id}/questions	Teacher	Body: text, options [{text, is_correct}]. Draft only. 201.
PUT /quizzes/{id}/questions/{qid}	Teacher	Replace text and options. Draft only.
DELETE /quizzes/{id}/questions/{qid}	Teacher	Draft only. 204.
POST /quizzes/{id}/publish	Teacher	Draft to Published.
POST /quizzes/{id}/close	Teacher	Published to Closed.
POST /quizzes/{id}/attempts	Student	Starts an attempt. 201 with attempt id and questions (no is_correct).
POST /attempts/{id}/submit	Student	Body: answers [{question_id, option_id or null}]. 200 with score, total, percentage.
GET /attempts/{id}	Student, Teacher	Student: own submitted attempt with per-question review. Teacher: attempts of own quizzes.
GET /attempts	Student	Own attempts, newest first.
GET /quizzes/{id}/results	Teacher	Summary (attempt_count, average_percentage, highest, lowest) and paginated attempts. Owner only.
GET /health	Public	Returns 200 {"status":"ok"}.

Ordering

- Quiz lists: created_at descending, then id. Attempt lists: submitted_at descending, then id. Questions and options: by position.

Request examples

Create quiz: {"title":"Python Basics","description":"Short quiz on Python fundamentals."}

Add question: {"text":"Which keyword defines a function?","options":[{"text":"def","is_correct":true},{"text":"func","is_correct":false},{"text":"lambda","is_correct":false}]}

Submit: {"answers":[{"question_id":"<uuid>","option_id":"<uuid>"},{"question_id":"<uuid>","option_id":null}]}

8. Errors and reliability

All errors use {"error":{"code":"...","message":"...","fields":{},"request_id":"..."}}. Unknown JSON fields are rejected (for example status, teacher_id, score in a request body).

HTTP	Code	Meaning
400	MALFORMED_JSON	Body cannot be parsed.
401	UNAUTHENTICATED	Missing, invalid or expired token.
403	FORBIDDEN	Role not allowed for this action.
404	NOT_FOUND	Missing or inaccessible resource.
409	INVALID_TRANSITION	Quiz status change not allowed.
409	QUIZ_LOCKED	Edit attempted on a Published or Closed quiz.
409	QUIZ_NOT_AVAILABLE	Attempt on a quiz that is not Published.
409	ATTEMPT_EXISTS	Student already attempted this quiz.
409	ATTEMPT_SUBMITTED	Attempt already submitted.
422	VALIDATION_ERROR	Invalid fields, answers, filters or publish rules.
500	INTERNAL_ERROR	Unexpected failure, no internal details.

- **API01** Two simultaneous start requests by the same student on the same quiz shall create exactly one attempt (database constraint decides; the other returns 409).
- **API02** Submit uses a database transaction. A repeated submit returns 409 ATTEMPT_SUBMITTED and does not change the stored score.
- **API03** Authorization is checked before validation and before any change.

9. Quality and operational requirements

ID	Area	Requirement
NF01	Security	Passwords hashed with bcrypt. JWT with expiry (60 minutes). Parameterized queries through the ORM. Rendered text escaped; script-like input stays inert. HTTPS outside local development. CORS restricted to the frontend origin.
NF02	Secrets	JWT secret and database URL come from environment variables, never committed. Provide .env.example.
NF03	Performance	With 1,000 quizzes and 20,000 attempts, p95 read responses are 1 second or less for 20 concurrent users.
NF04	Usability	All primary actions usable by keyboard and at 375 px and 1440 px widths.
NF05	Consistency	Attempt, answers and score commit in one transaction. A simulated failure leaves no partial result.
NF06	Observability	Log request ID, route, status and duration. Do not log passwords, tokens or answers.
NF07	Verification	Automated tests for validation, permissions, lifecycle and scoring. Smoke test for the Teacher and Student journeys.
NF08	Maintainability	Layered backend (routers, services, repositories). README with run and test instructions. Docker Compose for PostgreSQL.

Architecture boundaries

The React frontend calls the REST API. The FastAPI backend owns authorization, validation, workflow and transactions. PostgreSQL persists all data. Suggested libraries: SQLAlchemy, Alembic, Pydantic, python-jose, passlib (bcrypt), Pytest with HTTPX; Vite, React Router and Axios on the frontend; Jest with React Testing Library.

Suggested project structure

```
quiz-app/
  docs/SDD.md
  docker-compose.yml
  backend/
    app/ (main.py, core/, models/, schemas/, routers/, services/, repositories/)
    alembic/   tests/   seed.py   .env.example
  frontend/
    src/ (api/, auth/, components/, pages/)
  DECISIONS.md   README.md   TEST_RESULTS.md
```

10. Acceptance criteria and test cases

Seed one Teacher A, one Teacher B, Student A, Student B and one inactive account. Each test is mandatory. Verify database state as well as HTTP response and visible UI behavior.

Test	Given and action	Expected result and trace
TC01	Teacher creates quiz with valid title and description.	201, status Draft, owner is Teacher A. FR01, S07.
TC02	Create quiz with blank title, short description or unknown field (status).	422, field errors, nothing created. FR02.
TC03	Title lengths 4, 5, 150, 151; description 9, 10, 1000, 1001.	Only inclusive boundaries pass. Section 6.
TC04	Request without token; login as inactive user.	401; nothing created. P01, P05.
TC05	Student calls POST /quizzes.	403, nothing created. P02.
TC06	Teacher B reads, edits or views results of Teacher A's quiz.	404; no data exposed. P02, P03.
TC07	Teacher adds question with 3 options, 1 correct.	201. FR03.
TC08	Question with 1 option, 7 options, 0 correct or 2 correct.	422 each. FR03, Section 6.
TC09	Publish Draft quiz with 0 questions.	422; stays Draft. FR06.
TC10	Publish quiz with valid questions.	200, Published. FR06.
TC11	Edit question or title of a Published quiz.	409 QUIZ_LOCKED; unchanged. P07, FR04.
TC12	Delete Draft quiz; delete Published quiz.	204; 409. FR05.
TC13	Student lists quizzes.	Only Published quizzes, with attempted flag. FR07.
TC14	Student reads a Draft quiz by ID.	404. P02.
TC15	Student starts attempt on Published quiz.	201; questions without is_correct anywhere in the response. FR08, P04.
TC16	Student starts a second attempt on the same quiz.	409 ATTEMPT_EXISTS; one attempt in database. P08.
TC17	Student starts attempt on Closed or Draft quiz.	409 QUIZ_NOT_AVAILABLE (or 404 for Draft). FR14.
TC18	Submit 5 questions, 3 correct, 1 wrong, 1 unanswered.	200; score 3, total 5, percentage 60.0. FR09, FR10.
TC19	Submit with option from another question, or unknown question ID.	422; attempt stays In Progress. FR09.
TC20	Submit the same attempt twice.	Second returns 409 ATTEMPT_SUBMITTED; score unchanged. FR11, API02.
TC21	Submit body contains score field.	422 unknown field. P09.
TC22	Student B reads Student A's attempt.	404. P02.
TC23	Student views own result.	Per-question review with correct options. FR12.

Test	Given and action	Expected result and trace
TC24	Teacher views results with 3 attempts.	Correct count, average, highest, lowest. FR13.
TC25	Close Published quiz, then Student starts new attempt.	Close 200; start 409. Existing results stay. FR14.
TC26	Close a Draft quiz; publish a Closed quiz.	409 INVALID_TRANSITION. P06.
TC27	Two simultaneous start requests, same student and quiz.	Exactly one attempt. API01.
TC28	Simulate database failure during submit.	No answers or score stored; retry succeeds once. P10, NF05.
TC29	HTML or script text in title, question and option.	Stored as text, rendered inert. NF01.
TC30	Keyboard-only and narrow viewport run of both journeys.	All actions usable. UI06, NF04.
TC31	End-to-end: teacher builds and publishes, student attempts, teacher views results.	Full journey passes. S01 to S04.

Overall release acceptance

All S requirements and adopted P requirements trace to passing tests. Both end-to-end journeys pass. No open defect allows unauthorized access, leaked correct answers before submit, a second attempt, or a wrong score. Record environment, date and result for every test in TEST_RESULTS.md.

11. Delivery plan and decisions

Implementation sequence

- **Task 1 - Foundation:** repo structure, Docker Compose PostgreSQL, FastAPI skeleton with /health, React skeleton, Alembic migrations for all tables, seed script, login, JWT and role checks. Verify TC04, TC05.
- **Task 2 - Quiz and question management (backend):** all quiz and question endpoints with validation and lifecycle rules. Verify TC01 to TC12, TC26.
- **Task 3 - Teacher UI:** My Quizzes, Quiz Editor, publish, close, delete. Verify TC31 (teacher part).
- **Task 4 - Attempts and scoring (backend):** start, submit, results endpoints. Verify TC13 to TC25, TC27, TC28.
- **Task 5 - Student UI and results UI:** Available Quizzes, Take Quiz, Result Detail, My Results, Quiz Results. Verify TC31.
- **Task 6 - Hardening:** error states, accessibility, security checks, README, full test run, TEST_RESULTS.md, review note. Verify TC29, TC30.

Decisions for review

Decision	Proposed baseline	Effect if changed
Identity	Seeded accounts, email and password, JWT	Registration or SSO adds screens and tests.
Question types	Single-correct multiple choice	Other types change data model and scoring.
Attempts	One per student per quiz	Retries need attempt numbering and best-score rules.
Timing	No time limit	Timers need server-side expiry and auto-submit.
Answer visibility	Correct answers shown after submit	Hiding them needs a result visibility setting.
Reporting	Excluded	Needs separate requirements.

Specification change control

Store this document with the code in docs/SDD.md. For a requirement change: update the requirement, affected API or data contract and tests first, then the code. Record version, date and affected IDs. AI-generated code shall be reviewed against this document.

Review checklist

- Every S and P requirement is implemented and traced to a passing test.

- Business rules live in the service layer, not in routers or React components.
- Every endpoint has role checks and input validation.
- No secrets in the repository; migrations used for all schema changes.
- UI handles loading, empty and error states.

Appendix A. Prompt to give the AI agent

Copy this into your AI coding tool together with this document:

> *You are a senior full-stack engineer. Build the Online Quiz and Exam System exactly as specified in the attached SDD (docs/SDD.md), using React JS, FastAPI and PostgreSQL. Work through Tasks 1 to 6 in Section 11 in order. After each task, run the tests listed for that task and fix failures before continuing. Do not add features outside the SDD. Where the SDD is silent, choose the simplest option and record it in DECISIONS.md. Write automated tests for every test case TC01 to TC31, include a README with setup and run commands, and at the end produce TEST_RESULTS.md with a table of every test case, its result and notes, plus a short review note comparing the finished application to the SDD requirements.*

Appendix B. Document history

Version	Date	Change
1.0	29 Oct 2026	Initial draft specification